

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1

Software Design

Author: Ghile Patricia-Maria

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

March 2026

Contents

1	Problem definition	2
2	Analysis phase - Use cases & requirements	2
2.1	Functional requirements	2
2.2	Use cases	3
3	Design phase	3
3.1	Architecture overview	3
3.2	Entity-relationship diagram	3
3.3	Component: Controller Layer	4
3.4	Component: Service & Repository Layer (Model)	4
3.5	Class diagrams	4
3.6	Class diagrams	5
3.7	Sequence diagrams	5
4	Implementation phase - Application	6
4.1	Tools and technologies	6
4.2	Application structure	6
4.3	User manual	7
5	Testing and validation	7

1 Problem definition

The objective of this project is to implement an application that manages a specific domain entity, namely the Eurovision Song Contest catalog. The application is called **Eurovision Management System**.

The application supports multiple user roles (Admin, Head of Delegation, and Fan/Visitor) and allows users to interact with the system according to their strict permissions. The main purpose of this assignment is to build a simple but well-structured application using object-oriented principles and a layered architecture.

The following sections detail the development phases:

- **Analysis phase:** Defines the functional requirements and system use cases.
- **Design phase:** Details the MVC architecture and the database schema.
- **Implementation phase:** Describes the technology stack (JavaFX, PostgreSQL, JDBC) and UI.
- **Testing and validation:** Outlines the methods used to verify functionality and authorization.

Deliverables: Source code, PostgreSQL database schema, and this documentation.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

Functional requirements describe the operations that the system must perform. Based on the problem statement, the core functionalities include:

- **Authentication:** The system must allow users to log in.
- **Authorization:** The system must restrict access to specific features based on user roles.
- **Admin CRUD:** The Admin must be able to Create, Read, Update, and Delete any Eurovision entry regardless of the country.
- **HOD CRUD:** The Head of Delegation (HOD) must be able to Create, Read, Update, and Delete entries ONLY for their assigned country.
- **Search:** The system must allow users to search for specific entries using keywords.
- **Sort:** The system must allow users to sort the catalog by various criteria (e.g., Country, Artist, Year).

2.2 Use cases

Use cases describe the interactions between actors (users) and the system. The main actors in this system are the Admin, the Head of Delegation (HOD), and the Fan (visitor).

Below is the visual representation of these interactions:

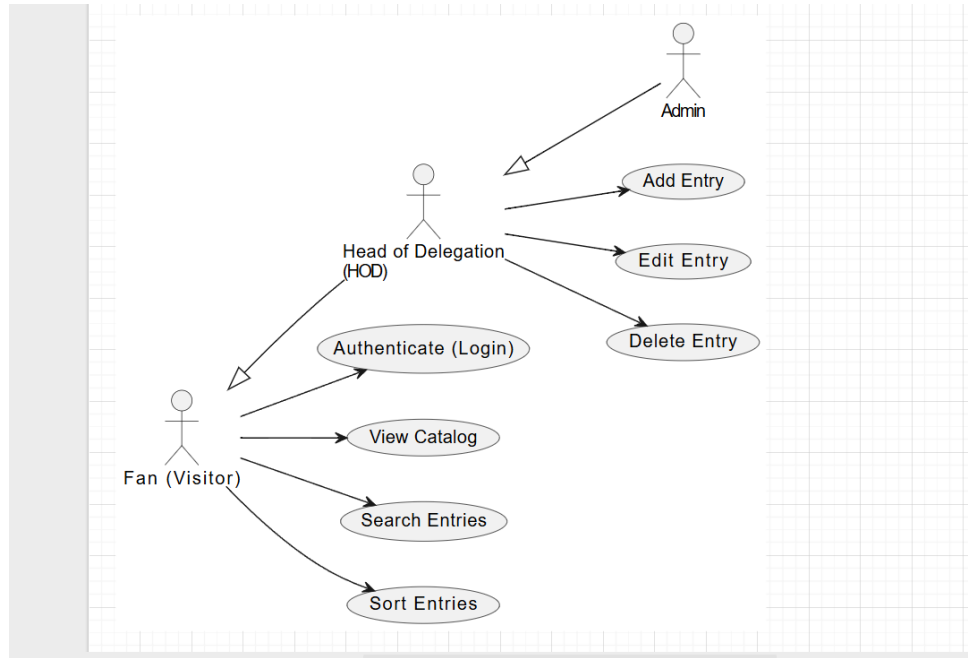


Figure 1: Use Case Diagram

3 Design phase

3.1 Architecture overview

The application follows the **Model-View-Controller (MVC)** architectural pattern combined with a layered approach. The system is structured as follows:

- **View:** Represents the graphical user interface, implemented using FXML files.
- **Model:** Represents the domain of the application and includes the domain entities, the repositories (data access components), and the business logic implemented in services.
- **Controller:** Controllers handle incoming user requests, coordinate the application flow, and delegate business logic to the service layer. Controllers do not contain any business logic or persistence logic.

3.2 Entity-relationship diagram

The database consists of two main tables: **users** and **entries**.

- **users:** Stores user credentials (username, password), roles, and the country they represent (for HODs).

- **entries:** Stores information about the songs (country, artist, title, release year).

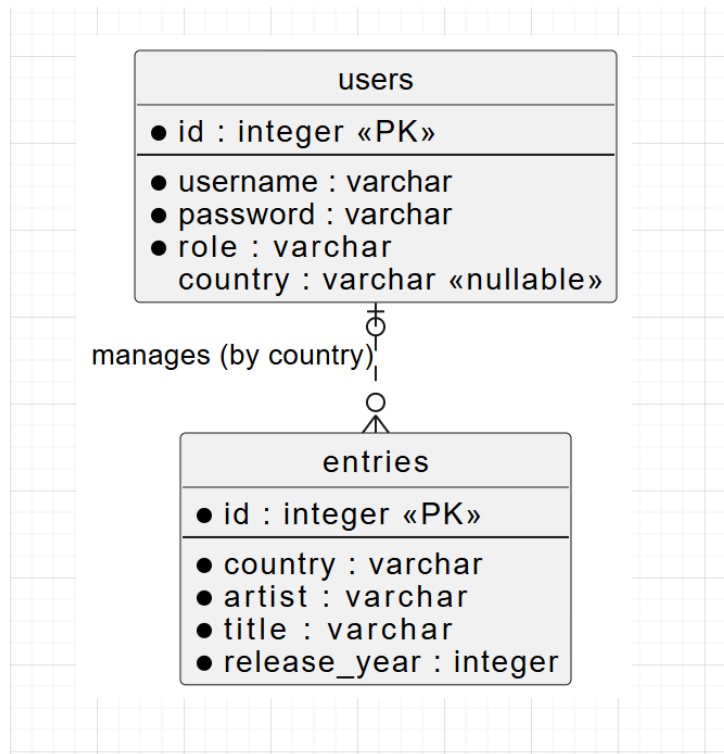


Figure 2: Entity-Relationship Diagram

3.3 Component: Controller Layer

This component manages the graphical user interface flow. Classes like **LoginController** and **EntryController** capture user inputs (button clicks, text fields) and pass the data to the Service layer. It is strictly responsible for updating the UI dynamically without executing any database or business operations directly.

3.4 Component: Service & Repository Layer (Model)

The **Service** classes contain the core business logic (e.g., verifying if an HOD has the permission to edit a specific country's entry). The **Repository** components handle the data access logic, executing the actual SQL queries (INSERT, SELECT, UPDATE, DELETE) via JDBC to communicate with the PostgreSQL database.

3.5 Class diagrams

The class diagram illustrates the static structure of the system, presenting the domain entities, repositories, services, and controllers.

3.6 Class diagrams

The class diagram illustrates the static structure of the system, presenting the domain entities, repositories, services, and controllers.

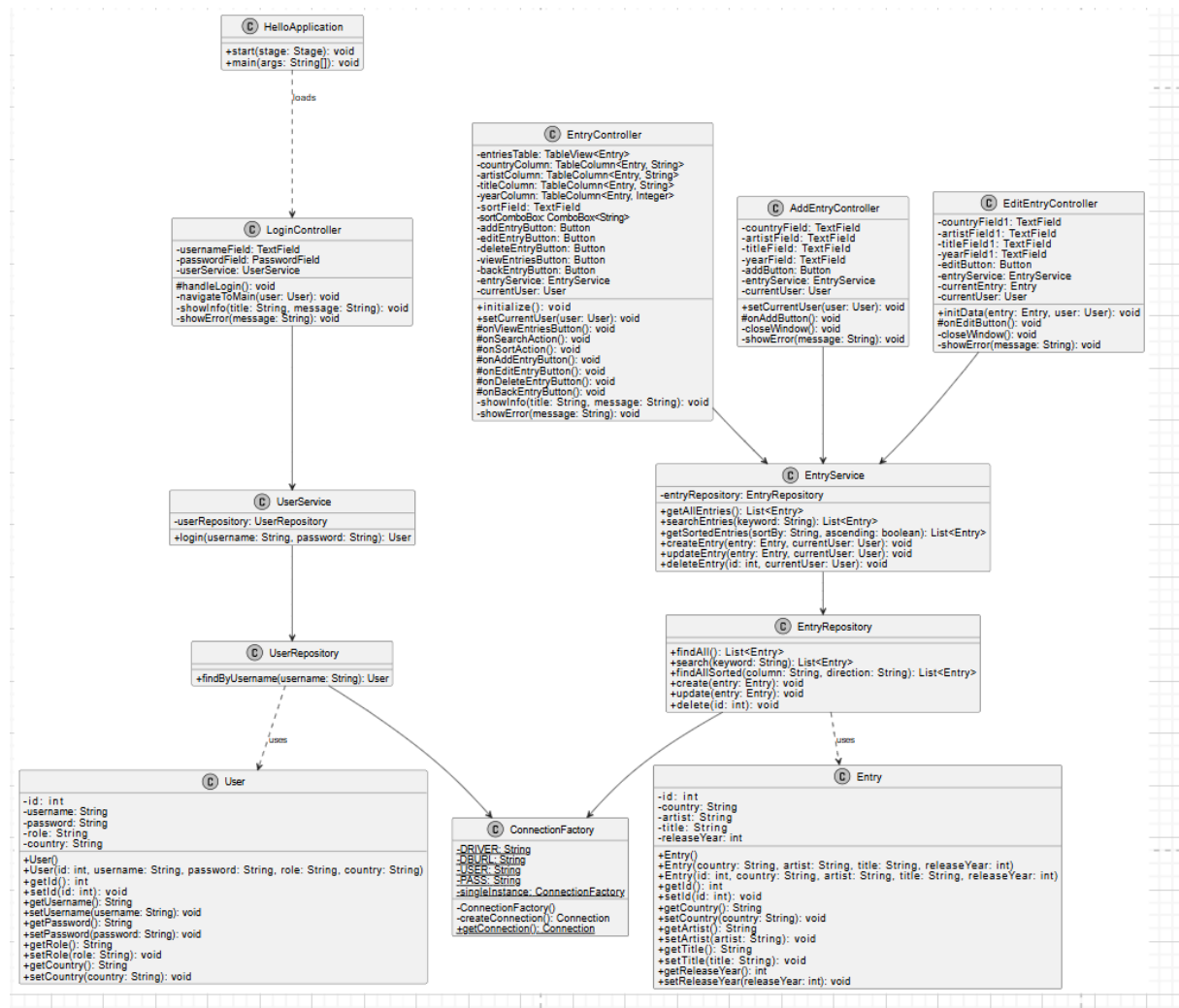


Figure 3: Class Diagram

3.7 Sequence diagrams

The sequence diagram below illustrates the flow of messages and method calls between system components during the authentication process, covering both successful login and error handling.

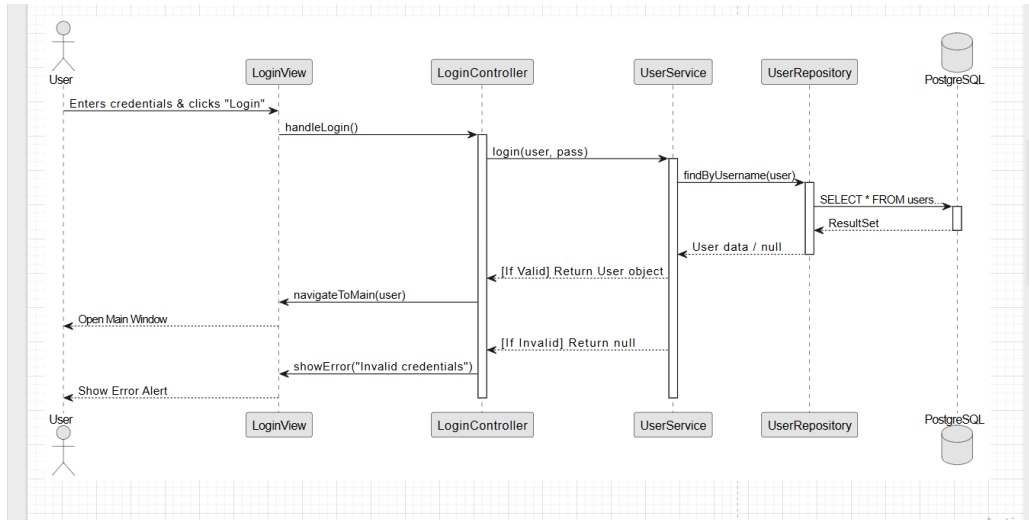


Figure 4: Sequence Diagram - Authentication Flow

4 Implementation phase - Application

4.1 Tools and technologies

The application was built using the following technology stack:

- **Programming language:** Java 21
- **GUI Framework:** JavaFX (with FXML and Scene Builder)
- **Database management system:** PostgreSQL
- **Database Connectivity:** JDBC API

Java and JavaFX were chosen for their robust object-oriented features and native desktop UI capabilities. PostgreSQL is a reliable relational database that integrates seamlessly with Java via JDBC.

4.2 Application structure

The source code is organized into specific packages to enforce the MVC layered architecture:

- `org.example.eurovision.manager.controller`: Contains UI logic and controllers.
- `org.example.eurovision.manager.model.entity`: Contains domain entities (User, Entry).
- `org.example.eurovision.manager.model.repository`: Contains data access components.
- `org.example.eurovision.manager.model.service`: Contains the business logic.
- `view`: Contains the FXML files representing the View layer.

4.3 User manual

Upon launching the application, the user is greeted by the **Login Window**. The user must input their username and password to authenticate.

Upon successful login, the **Main Window** opens. The interface and available actions adapt dynamically based on the user's role:

- **Admin:** Has full access to the "Add", "Edit", and "Delete" buttons and can modify entries for any country.
- **Head of Delegation (HOD):** Has access to the action buttons but can only add, edit, or delete entries that belong to their assigned country.
- **Visitor (Fan):** The action buttons are completely hidden. The user has a read-only view of the catalog.

All users can use the search text field to filter entries by keyword, or the combo box to sort the table dynamically. A "Back" button allows the user to log out and return to the login screen.

5 Testing and validation

The application was validated through rigorous manual UI testing, business logic verification in the **EntryService** layer, and database consistency checks. All possible interaction scenarios were tested to ensure that the system satisfies the requirements defined in the analysis phase.

- **Role-Based Access Control (RBAC):** Verified that the interface adapts correctly to each user role. For the **Fan** role, all modification buttons (Add, Edit, Delete) are hidden. For **Admin** and **HOD**, the buttons are visible, but the underlying service layer enforces strict access rules.
- **Business Logic & Permissions:** Manually tested the logic in **EntryService** to confirm that:
 - The **Admin** can successfully perform CRUD operations on any entry in the catalog.
 - An **HOD** can only add, edit, or delete entries belonging to their specific assigned country. Any attempt to modify data from another country is correctly intercepted by the service layer, which throws a **RuntimeException** and displays a clear error message to the user.
 - All permission-based restrictions work as intended, and no unauthorized actions can reach the database.
- **Database Consistency:** Verified through the PostgreSQL console that all CRUD operations performed via the JavaFX interface are correctly persisted in the **entries** table.
- **Authentication & Error Handling:** Tested multiple login scenarios, including empty fields, incorrect credentials, and valid users. The system provides appropriate feedback and navigates correctly only upon successful authentication.

In conclusion, all functional cases and permission scenarios were thoroughly tested, confirming that the application behaves correctly according to the specified business rules and roles.

References

- [1] Oracle. *Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification*. <https://docs.oracle.com/en/java/javase/21/docs/api/index.html> (Accessed: May 2024).
- [2] OpenJFX. *JavaFX Documentation - Getting Started*. <https://openjfx.io/openjfx-docs/> (Accessed: May 2024).
- [3] PostgreSQL Global Development Group. *PostgreSQL 16.0 Documentation*. <https://www.postgresql.org/docs/16/index.html> (Accessed: May 2024).
- [4] Gluon. *Scene Builder User Guide*. <https://gluonhq.com/products/scene-builder/> (Accessed: May 2024).
- [5] Reenskaug, T. *The Model-View-Controller (MVC) Architectural Pattern*. Software Engineering Institute, Carnegie Mellon University.
- [6] Oracle. *JDBC Database Access*. <https://docs.oracle.com/javase/tutorial/jdbc/> (Accessed: May 2024).